

DSecO: Domain Name System (DNS) Data as a Knowledge Graph for Enhanced Security Analysis

Didier Bringer^{ID} and Lionel Tailhardat^{ID}

Abstract—Managing Domain Name System (DNS) records presents challenges in terms of consistency and tracking over time, which can have operational impacts, particularly on cybersecurity. In this paper, we explore the use of ontologies and knowledge graphs to facilitate DNS system audit activities. We define nine key use cases derived from real DNS operations data of a large-scale telco company and demonstrate how querying and inference techniques can address these use cases using an RDF knowledge graph structured by the DSecO vocabulary, an open-source ontology available at <https://w3id.org/dseco>. Overall, we demonstrate the feasibility of providing a unified view of DNS records and the practicality of explainable and shareable DNS administration rules for extended analysis and informed decision-making by NetOps and SecOps teams regarding DNS records.

Index Terms—DNS audit, network operations, cybersecurity, ontologies, knowledge graphs, semantic web, AXFR, domain hijacking.

I. INTRODUCTION

ON THE Internet, a primary address such as `www.example.org` – referred to as a fully qualified domain name, or FQDN [13, §2.3.5] – must resolve to one or more final IP addresses (either IPv4, IPv6, or both). The FQDN to IP lookup is a key mechanism, commonly implemented using a Domain Name System (DNS) [40], to determine how to connect to network services (e.g. the host serving Web pages for `www.example.org`). Organizations managing large-scale networks, such as international corporations or Internet service providers, may utilize the DNS CNAME (Canonical Name) [40, §3.3.1] functionality (Listing 1), for example to handle the complexity and constant evolution of the network infrastructures and services they operate by defining generic addresses. However, this constant evolution also requires ongoing adjustments of DNS records by NetOps and SecOps teams, both to ensure the proper functioning of services and to prevent cybersecurity issues.

For instance, the CNAME example of Listing 1 summarizes as a rule chain akin to `www.example.org` $\xrightarrow{\text{CNAME}}$ `www.example.myotherzone.org` $\xrightarrow{A}$ `203.0.113.1`, where `www.example.myotherzone.org` should be

Received 14 January 2025; revised 3 July 2025; accepted 8 August 2025; approved by IEEE TRANSACTIONS ON NETWORKING Editor E. Bertino. Date of publication 11 September 2025; date of current version 30 December 2025. This work was supported in part by the Orange - Fonds d’Innovation pour la Sécurité (FSI) led by Jean Marc Blanco. (Corresponding author: Lionel Tailhardat.)

The authors are with Orange Company, 92449 Issy-les-Moulineaux, France (e-mail: lionel.tailhardat@orange.com).

Digital Object Identifier 10.1109/TON.2025.3598374

Listing 1 Example of DNS records with a CNAME clause: CNAME allows one domain name to alias another for address redirection. For instance, when a user attempts to access `www.example.org`, the operating system converts it to `203.0.113.1`, enabling the browser to connect to the server with this IP address and retrieve the requested page.

```
1 # In the example.org zone:
2 www.example.org. CNAME www.example.myotherzone.org.
3
4 # In the myotherzone.org zone:
5 www.example.myotherzone.org. A 203.0.113.1
```

removed for a complete cleanup when `www.example.org` is no longer in use. It is noteworthy that the CNAME of CNAME pattern (known as a CNAME chain) – although not recommended¹ – can still be found in many DNS records.²

Stale DNS records can also pose cybersecurity risks. For example, DNS record chains that include a DNS record part of a bankrupt Web agency’s DNS zone can lead to a “domain hijacking”, as hackers could purchase the agency’s domain name and receive requests made to `www.example.org`. This can result in credential leaks, as the user’s Web browser may send cookies to the hackers’ server. While HTTPS and HSTS [20] help reduce this risk, it remains high due to automated and free Certificate Authorities (CAs) like Let’s Encrypt.³ Indeed, hackers can obtain valid certificates in someone else’s name and succeed the identity verification challenge by simply placing a nonce file on the target server they control.⁴

These examples highlight the numerous rules needed to analyze DNS records, which can be tedious and error-prone when done manually within a given organization, especially for medium to large-sized companies with thousands to tens of thousands of records and complex interconnections.

Automating DNS record analysis can enhance operational efficiency, prevent incomplete cleanups and security vulnerabilities, and improve overall DNS service performance by reducing lookup operations [6]. However, challenges in data integration and knowledge representation arise due to the sharing of DNS data across different zone owners and heterogeneous systems. Indeed, providing a unified view of DNS

¹CNAME chains increase DNS server load, cause name resolution delays [6], and expand the trusted computing base (TCB), posing security risks [45].

²Particularly in CDNs to improve content delivery efficiency [50].

³<https://letsencrypt.org/>

⁴CAA DNS records [42] can mitigate this.

records, potentially linked to network infrastructure and business data, is essential for informed decision-making by NetOps and SecOps teams for both DNS administration tasks and day-to-day monitoring of the networks behavior. Additionally, establishing explainable and shareable DNS administration rules is crucial to ensure a clear understanding of their execution and mutual agreement among zone owners.

In this work, we hypothesize that an explicit graph representation could reflect the chaining nature of DNS records and provides formal guarantees for various use cases. To tackle the above-mentioned challenges, we propose the “DNS-KG” approach, which involves building an RDF knowledge graph [1] from heterogeneous DNS data within an organization to facilitate cross-zone analysis of DNS records. The graph is structured by DSecO, a lightweight ontology we have implemented in RDFS [9] / OWL [54] to enable a query-based audit approach of DNS data and facilitate connections to third-party knowledge bases for broader inference cases, such as linking DNS configuration to Cyber Threat Intelligence (CTI) data [24], [56]. We formalize nine DNS administration use cases based on interviews with DNS operation experts from Orange – a leading international network infrastructure and service provider – and implement them as SPARQL queries [55] to evaluate the query-based audit approach on both generated and real-world data. The DSecO implementation, evaluation dataset, and the associated documentation are available as open source at <https://w3id.org/dseco>. Overall, we demonstrate the feasibility of providing a unified view of DNS records and the practicality of explainable and shareable DNS administration rules for extended analysis and informed decision-making by NetOps and SecOps teams regarding DNS records.

The remainder of this paper is organized as follows. In Section II, we review related work, focusing on DNS-related security methods and tools, as well as semantic models for knowledge representation. In Section III, we provide details of the DNS-KG approach with a deep dive into the use case definitions, domain modeling with DSecO, and knowledge graph construction and exploitation for DNS records audit. In Section IV, we present the experiments conducted and their results on the DNS-KG approach, including running SPARQL queries related to the use case definitions. Finally, we conclude and outline some future work in Section V.

II. RELATED WORK

Various studies have explored DNS-related security methods and tools to detect vulnerabilities or malfunctions due to non-compliance or discrepancies in DNS, using either a glass-box or closed-box strategy on medium to large-scale systems (e.g. academic or corporate networks, the Internet).⁵ However, few studies examine this field through the lenses of knowledge representation and reasoning (KRR) and linked open data principles [1], even though several existing semantic models could be applied for explicit representation and reasoning on DNS configurations (e.g. UCO [56] and ICAS [31], further

discussed below). Thus, there is an opportunity to bridge *DNS-related security methods and tools* with *semantic models*. In the following paragraphs, we review existing works on these two aspects and then conclude with the positioning of our DNS-KG proposal.

a) *Glass-box methods & tools*: GRooT [50] and Heracles [25] are two methods aiming at verifying DNS configurations, with GRooT apparently the first formal approach in this field of study. Both methods are able to capture the entirety of DNS behavior within a graph representation (a DNS configuration interpretation graph), encompassing the transformation of DNS zone files into graphs to enhance the consistency verification process. This verification can be performed as either a self-check or a cross-check. In GRooT, users can implement new analysis cases as C++ functions, ensuring high performance in processing the interpretation graph. However, developing additional use cases (e.g. identifying risky or useless DNS entries) requires abstraction and programming skills from SecOps teams. Additionally, the specific data structure of the interpretation graph and the static nature of analysis functions make it difficult to extend graph data with external sources or incorporate new data objects at runtime for non-monotonic reasoning. In the same vein as GRooT and Heracles, Moura et al. [18] developed and open-sourced the CycleHunter framework to analyze data from DNS zone files. Their work addresses a specific issue in DNS records: cyclic dependencies, highlighting the significant associated risk (i.e. a potential collapse of the entire Internet), which they refer to as a TsuNAME, and proposing methods for its remediation.

Regarding tools, the *dns-graph*⁶ project (a small GitHub project with no updates since 2018) introduces a database model for HOST, MX, DOMAIN and NS records but does not take CNAME records into account. The overall approach involves populating a Neo4J⁷ graph database with data and subsequently execute queries. Another interesting project is *PowerDNS*⁸ which implements a DNS system that can integrate a database backend and provide an API, facilitating queries compared to traditional file-based DNS systems. However, it only queries zones under its delegation.

b) *Closedbox methods & tools*: Ramasubramanian et al. [45] surveyed nearly 600’000 Web server names to establish a dependency graph of authoritative name servers. For a given Web server name, they highlighted that a larger Trusted Computing Base (TCB) increases risk, as compromising any name server within the delegation graph can jeopardize the name. They also compiled known vulnerabilities of public name servers to perform a risk assessment. While this study provides a valuable overview of name system vulnerabilities, it does not provide details about how the data was stored. Also on the topic of trust, Daiping et al. [8] demonstrated that subdomains inherit the trust of their apex domains, which is particularly valuable for illicit online activities. Indeed, the theft of Fully Qualified Domain Names (FQDNs) through IP hijacking or domain hijacking – cases tackled by our DNS-

⁵Glass-box testing analyzes a system’s internal workings with full access to its source code, focusing on identifying vulnerabilities, while closed-box testing evaluates functionality from an external perspective, simulating user interactions without knowledge of the internal code.

⁶<https://github.com/abhisek/dns-graph>

⁷<https://neo4j.com/>

⁸<https://github.com/PowerDNS/pdns>

KG approach – can lead to a variety of serious and detrimental consequences for the brand that owns the domain.

More focused on DNS record management, the authors of [7], [15], [17], and [26] all highlight the often-overlooked severity of dangling or stale DNS records. Since they operate in a closed-box manner, they use passive DNS traffic data or brute force techniques to identify domains, followed by live testing to ascertain DNS responses. They all conclude that maintaining one's DNS properly over time is essential for security. In [44], DNS configuration updates are also discussed through the study of orphan and abandoned records in the Top-Level Domains (TLDs) zone file. The conclusion remains consistent across this body of work: mitigating related cybersecurity risks involves the process of cleaning areas.

Finally, regarding tools, the *DNSDumpster*⁹ project is a free online solution for DNS reconnaissance and domain information gathering. It offers insights into a domain's DNS records, identifies (some of the) subdomains, and visually represents the domain's DNS structure. It is commonly used by security professionals and researchers for network analysis and vulnerability assessments. Focusing more on facilitating navigation and analysis of the Internet data space through semantic integration of various data sources, Fontugne et al. [46] have developed and open-sourced the Internet Yellow Pages (IYP) project, which integrates 46 publicly available datasets into a Neo4j database. The project is designed to be community-driven, allowing dataset curation and data model evolution with provenance metadata (e.g. data source, collection date, update date).

c) *Semantic models*: In this paragraph, we briefly describe works involving semantic models in the form of RDFS/OWL ontologies. In direct relation to the field of cybersecurity, UCO [56] is a popular community-developed ontology built around the cyber security domain, covering a wide range of important concepts such as agents, resources, or actions. Original motivations for this work were knowledge discovery and After Action Report (AAR) annotation tasks. The ICAS ontology [31] closely resembles UCO in both structure and intent: a lengthy enumeration of objects that are more or less related and grouped by discourse domain for networking and cybersecurity. Although ICAS has interesting concepts and relationships to represent DNS configuration data, its structure seems overcomplicated and no example of implementation in a knowledge graph is available. Further, it is no longer maintained. Similar to the idea of leveraging CTI data in ontologies like UCO and ICAS, the authors in [5] propose an anomaly detection system that utilizes a combination of a SWRL-based [19] inference rule set and SPARQL queries to classify and provide guidance on various situations. Although this work has been evaluated on sensor data and does not directly relate to DNS configuration data, the principles developed for situation classification using reasoning can be generalized to other application domains. In a similar vein, authors in [24] introduce the ICS-SEC ontology to enable threat intelligence exploration, as well as vulnerability assessment and remediation use cases to the field of Industrial Control Systems (ICSs).

To achieve this, an ICS-SEC knowledge graph is constructed by aggregating several well-known security databases and data models, such as ATT&CK¹⁰ and NVD.¹¹ Like the work in [5], the proposed solution does not directly relate to DNS configuration data, but principles and technique for situation classification can generalize to side application domains.

Still in relation to cybersecurity, but focusing on software, Taghavi asserts in [37] that the rapid increase in vulnerabilities across various types of assets has rendered the synthesis of this knowledge inherently more challenging. In this context, knowledge graphs and their accompanying technology stack have been advocated as a viable solution for modeling, integrating, and facilitating interoperability among diverse data sources. We note that, surprisingly, Taghavi does not leverage the Software Bill of Materials (SBOM)¹² inventory to model the dependencies between components. Further, Taghavi did not make available any open source ontology from his work. Going a same line of thoughts, Alqahtani et al. introduced SECONT and MAVON [53], ontologies covering software artifacts and ontologies for software vulnerability databases. Another work, the Software Evolution Ontologies (SEON) [35], enables to bootstrap a modeling of GitLab activities, and thereby track code quality evolution over time, however no direct link to the analysis of vulnerabilities is available through SEON.

Finally, in relation to representing network infrastructures, the DevOpsInfra [39] ontology describes sets of computing resources and how they are allocated for hosting services. However, the ontology mostly focuses on the provisioning activity and concepts are missing for risk assessment and a finer grain description of the network topology. In close relation, NORIA-O [30] allows to describe an infrastructure, its events, and diagnosis and repair actions performed during incident management. NORIA-O connects to UCO, enabling reasoning about network topology and cybersecurity aspects simultaneously; however, it does not include concepts or relationships for DNS data.

d) *Dns-KG positioning*: The work presented in our DSecO paper globally aligns with the high-level objectives of the related works studied above, in that the DNS-KG approach aims to provide a method and tools for risk assessment and remediation for network service infrastructures heavily relying on DNS. For instance, several studies propose defining and executing assessment cases in a glass-box manner on a graph-based representation of DNS configurations (e.g. GRooT [50], Heracles [25], CycleHunter [18], and *dns-graph*).

However, works such as GRooT and Heracles conduct analyses to identify inconsistencies by deducing DNS behavior based on the subtleties of the domain's RFCs as interpretation rules, whereas DNS-KG focuses on analyzing the DNS "tree" itself to detect both risky and redundant entries, capturing CNAME and A records in a straightforward manner, without considering Time to Live (TTL) values or additional features. For example, TTL is a concept that refers to the duration

⁹<https://dnsdumpster.com/>

¹⁰<https://attack.mitre.org/>

¹¹<https://nvd.nist.gov/>

¹²<https://www.cisa.gov/sbom>

for which DNS data is considered valid, and because DNS-KG collects data directly from zone files rather than from an authoritative DNS server, TTL is not applicable at all as it would be in a closed-box approach. In our glass-box methodology, entries possess an infinite TTL, remaining valid as long as they exist in the zone file. Despite these differences in approaches, GRooT and Heracles do have points of convergence with DNS-KG in that their research has already contributed one use case in this paper (see `uc_loop` in Section III-A), which is a scenario likely to occur over time.

While we acknowledge that the DNS-KG structured by DSecO (Section III-B) does not capture all DNS features, we believe it is sufficient for DNS operations and administration use cases, starting with the nine defined with SecOps experts in Section III-A. For example, although DSecO currently does not implement NS records – such as for preventing TsuNAME attacks [18] – it does include the CNAME concept, which allows the analysis of CNAME chains and the detection of CNAME-based cyclic dependencies, among other things. It should be noted that the decision to implement specific concepts generally depends on the objectives of the projects studied, such as IYP [46], which, for example, chose not to include non-resolvable CNAMEs. Furthermore, DSecO can easily be extended with new concepts (e.g. a NS concept to materialize records already at hand in DNS zone dumps) and by linking the DNS-KG to other knowledge bases based on linked-open data principles (e.g. by adding vulnerability information about CTI informations [5], [45]; network informations [46]; SBOM information [37], [53]) to broaden the scope of risk assessment inference, thereby refining the risk categorization and remediation.

The DNS-KG approach also aligns with the need to clean DNS configurations regarding cybersecurity risks, as proposed by Borgolte et al. [26] and in [17]. However, while these works emphasize systematizing DNS operations using public data, the DNS-KG proposal adopts a glass-box strategy to secure dangling records on managed systems, including zones not directly under our control (unlike *PowerDNS*), provided we have access to the configuration data in some form.

Finally, we also observe that, compared to works such as GRooT, Heracles, *dns-graph*, and IYP, the DNS-KG approach brings a no-code approach to the field by leveraging RDF/RDFS/OWL syntax and vocabularies for the knowledge graphs and the DSecO ontology. For example, while knowledge graph construction and DNS assessment use cases could be implemented using programming languages (e.g. script-based data integration and Cypher¹³ queries in IYP, C++ functions in GRooT), the DNS-KG approach aims to enable users to build the graph using a declarative data integration approach (making it portable and logically auditable) by utilizing the RDF Mapping Language (RML) [3] vocabulary and performing assessments through SPARQL [55] queries.¹⁴ We also opted for RDF knowledge graphs, rather than labeled property graphs (LPGs) (as in [18], [46], and *dns-graph*). LPGs are oriented towards performance and flexibility for sim-

ple relationships and rich properties, whereas RDF knowledge graphs facilitate handling heterogeneous data and emphasize semantics and reasoning for complex relationships and interconnected knowledge, thanks to shared explicit knowledge representations [1]. Furthermore – in contrast with [37] and [45] – our work clearly articulates the approach to data storage and the methods of processing it, and is fully open-source.

Overall, we note that the DNS-KG approach addresses a gap in the aforementioned related works by providing explainable risk assessment over DNS configuration data and being open to including all data objects and relationships pertinent to security concerns (such as machines, Docker images, CaaS,¹⁵ PaaS,¹⁶ HTTP L7, organizations, CVEs,¹⁷ CPEs,¹⁸ and more).

III. METHODOLOGY—THE DNS-KG APPROACH

In this section, we introduce our DNS-KG approach to facilitate DNS administration operations – such as the cleanup of outdated records or those that pose cybersecurity risks – by leveraging an explicit representation of DNS record data as a knowledge graph. We assume that this representation can capture the chaining nature of DNS records and provide formal guarantees for various use cases through graph traversal and subsumption based on an ontology (i.e. the explicit representation of a discourse domain through concepts and relationships). Figure 1 summarizes the DNS-KG approach.

First, in Section III-A, we focus on operation support by identifying and formalizing nine key DNS administration use cases based on the analysis of DNS administration operations with SecOps experts from Orange. We use a Behavior-Driven Development (BDD) methodology to describe these use cases in natural language, making them suitable for automated data processing. Next, in Section III-B, we introduce the DSecO vocabulary, an open-source RDFS/OWL ontology that structures DNS-related data from various sources and enables reasoning related to the identified use cases. In Section III-C, we detail the data processing pipeline for constructing the knowledge graph using a no-code approach, and how it is queried by translating use cases into SPARQL queries to identify and classify potential non-compliance in DNS data. We present the related experiments and results in Section IV.

A. Use Cases Definition

To provide more detail about the analysis to be performed, we conducted a survey of DNS administration operations with a panel of SecOps experts from Orange. This panel consists of 12 operations specialists from the security department, who collectively represent over 20 years of experience managing more than 20'000 DNS records for Orange. Based on this survey, we define the following nine use cases, that will serve as a reference for the utilization of the knowledge graph developed in Section III-C. For each use case, we provide a brief description of the situation to monitor, and its formalization in Gherkin syntax [51]. The names of the

¹³Cypher is the query language for Neo4J.

¹⁴SPARQL is a standardized, widely used query language for RDF in both academia and industry.

¹⁵Container as a Service.

¹⁶Platform as a Service.

¹⁷CVEs: Common Vulnerabilities & Exposures.

¹⁸CPE: Common Platform Enumeration.

Listing 6 The `uc_entries_using_our_CDN` Gherkin model.

```

1 Given a DNS-KG
2 When a FQDN exists in the example.org zone
3   and has an (intermediary or final) FQDN
4   that is part of our CDN
5 Then focus on it

```

- f) *uc_resolving_to_a_single_ip_address*: Depending on your architecture, you may wish to identify the entries that resolve to a single IP address. If you are not utilizing a Virtual IP (VIP),¹⁹ it may be beneficial to recognize that these entries are not redundant.

Listing 7 The `uc_resolving_to_a_single_IP_address` Gherkin model.

```

1 Given a DNS-KG
2 When a FQDN exists in the example.org zone
3   and resolves in the end to an IP address
4   and the IP address is unique
5 Then focus on it

```

- g) *uc_domain_hijacking*: An entry on `example.org` directs to a Web agency; if the Web agency goes bankrupt, the domain may become available for purchase. A black-hat hacker might attempt to buy it and thus receiving the traffic. The hacker may take advantage of the domain name for phishing activities or potentially for distributing code to clients or the loading of JavaScript scripts. Alternatively, the hacker could create a webpage on this domain that redirects users to a site mimicking the appearance of `example.org`, using their own certificate, in an attempt to deceive users into entering their credentials. However, there exists a potentially more perilous scenario in which a hacker generates an SSL certificate in the name of `example.org`, utilizing certificate providers that authenticate the identity of the domain owner merely by requesting a nonce file to be placed on the website. Nevertheless, the risk of domain hijacking – an “Acquire Infrastructure: Domains”²⁰ technique as per MITRE ATT&CK – may be reduced due to the widespread use of HTTPS and HSTS [20] as well as CAA DNS records [42]. Identifying these domains is crucial for security, as indicated by the action “Then it should be monitored closely” in Listing 8. This means that NetOps/SecOps must manage the risk of domain hijacking by obtaining guarantees from the provider or thorough pentesting.

- h) *uc_ip_hijacking*: An entry on `example.org` directs to an IP address that is outside the company’s subnet and belongs to another Internet Service Provider (ISP). A black-hat hacker might attempt to obtain the same IP by purchasing services from that ISP. Borgolte et al. [26] call these “IP address use-after-free vulnerabilities”

Listing 8 The `uc_domain_hijacking` Gherkin model.

```

1 Given a DNS-KG
2 When a FQDN exists in the example.org zone
3   and has a intermediary or final FQDNs
4   in a zone which is not managed
5   by ORG public nor ORG internal
6 Then it should be monitored closely

```

and MITRE ATT&CK categorizes them under “Acquire Infrastructure: Server”.²¹

IP hijacking may permit the attacker to obtain a valid certificate for the domain, but even once the domain RR is fixed, it may lead to a security risk as the hacker can use the SSL certificate in a man-in-the-middle attacks in a closed network, such as a coffee shop [26].

Listing 9 The `uc_ip_hijacking` Gherkin model.

```

1 Given a DNS-KG
2 When a FQDN exists in the example.org zone
3   and resolves in the end to an IP address
4   and the resolved IP address is outside of our subnet
5 Then it should be monitored closely

```

- i) *uc_ip_hijacking_filter*: This use case derives from the *uc_ip_hijacking*, connecting the AS, subnet, and organizational units concepts to filter the results more strictly. We assume that a list of friendly AS organizations is available, enabling to focus on other AS (i.e. non-friendly organizations are to be scrutinized).

Listing 10 The `uc_ip_hijacking_filter` Gherkin model.

```

1 Given a DNS-KG
2 When a FQDN exists in the example.org zone
3   and resolves in the end to an IP address
4   and the resolved IP address is outside of our subnet
5   and this IP address is in a subnet
6   which is declared by an AS
7   and this AS has an Organization
8   which is not whitelisted
9 Then it should be monitored closely

```

B. Domain Modeling With DSecO

In relation to RFC 1035, which describes the details of the domain system and protocol (see particularly [40, §3]), we identify the essential concepts and relationships from the above Gherkin models (Listings 2 to 10) to establish the necessary terminology for the DNS-KG. For example, analyzing the *uc_complete_cleanup* case of Listing 2 leads us to the following axiomatisation (Eq. 1) as a first approach:

$$\begin{aligned}
 \text{ToRemove} \equiv & (FQDN_{end} \sqcap \text{managedBy.Org}_{int}) \sqcap \\
 & \neg (FQDN_{top} \\
 & \quad \sqcap \text{linkedTo}^+.FQDN_{end} \\
 & \quad \sqcap \text{managedBy.Org}_{top})
 \end{aligned} \tag{1}$$

We choose to implement this terminology in the form of an RDFS/OWL ontology to address knowledge representation

¹⁹VIP: an IP address not tied to a specific physical interface, used for load balancing, failover, or redundancy in networking.

²⁰<https://attack.mitre.org/techniques/T1583/001/>. Includes domain takeover.

²¹<https://attack.mitre.org/techniques/T1583/004/>

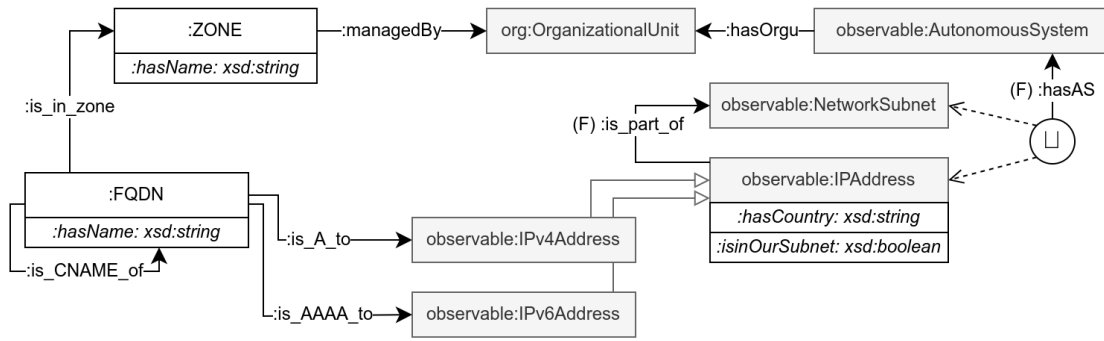


Fig. 2. The DSecO ontology. This diagram provides an overview of the ontology using the Chowl notation [49]. The base namespace is `dseco`, which URI is <https://w3id.org/dseco/ontology/>. Additional namespaces are: `observable` = <https://ontology.unifiedcyberontology.org/uco/observable/>, `org` = <http://www.w3.org/ns/org#>, `owl` = <http://www.w3.org/2002/07/owl#>, and `xsd` = <http://www.w3.org/2001/XMLSchema#>. The shaded forms highlight the concepts and properties of third-party vocabularies reused in DSecO, while the others are defined by DSecO.

and reasoning (KRR) challenges, particularly regarding the management of heterogeneous data sources (e.g. data fetched from various DNS sources, combined with data from BGPKIT [38] for subnet/AS number (ASN) [22] relationships, and internal databases for zone ownership) and ensuring linked and interoperable data representation (e.g. associating DNS-KG entities with cyber threat intelligence [24], [37], [56] knowledge graphs or LDAP²² directory data [48]). In terms of ontology design principles, we also have considered the RFCs as a normative information source for the rules of transforming DNS configuration data into a knowledge graph representation; the principle here is to prioritize understanding among users by adhering as closely as possible to the standard to minimize data wrangling and interpretation efforts back and forth between the DNS-KG and the source data. To further enhance usability, we also considered essential to hide the complexity of the use cases-related SPARQL queries (Section III-C), making the DNS-KG system more accessible. Additionally, minimizing graph traversal times is crucial for improving performance (e.g. for closure calculus on CNAME chains).

As a result, we have implemented the DSecO conceptual model which – for its release v1.6.1 – consists of two classes (FQDN, ZONE), eight objects properties (hasAS, hasOrgu, is_A_to, is_AAAA_to, is_CNAME_of, is_in_zone, is_part_of, managedBy) fully mapped to the RFC 1035, and four data type properties. These elements are defined in the `dseco` namespace. The ontology connects to the well-known **ORG** [10] and **UCO** [56] vocabularies, thereby enabling connections of the DNS-KG to third-party knowledge bases and adhering to Linked Open Terms (LOT) principles [32]. The DSecO expressivity is $ALURF(D)$ as per Protégé 5.1.²³ Figure 2 gives an overview of the ontology. Its implementation is directly available online at <https://w3id.org/dseco/ontology/> using an ontology browser, and its documentation at <https://w3id.org/dseco/doc/>.

In this paragraph, we introduce the DSecO concepts and properties. We define the `dseco:FQDN` class for representing fully qualified domain names (e.g. `www.example.org`) as individuals, with domain names attached as strings via the `dseco:hasName` property (e.g. for text-based searches).

The `dseco:is_CNAME_of` property links FQDN entities to represent FQDN aliases – typically expressed as DNS CNAME records, as shown in Listing 1. FQDN to IP reasoning is supported by the `dseco:is_A_to` property, which ranges to the UCO `observable:IPv4Address` class used to describe DNS A records [40, §3.4.1], such as `www.example.myotherzone.org`. A `203.0.113.1` in Listing 1. Reusing the UCO `observable:IPAddress` class within DSecO facilitates interoperability of the DNS-KG with third-party CTI-related knowledge bases. Similarly, we define the `dseco:is_AAAA_to` property with range `observable:IPv6Address`. From UCO, we also reuse the `observable:NetworkSubnet` class, and define the `dseco:hasAS` property with range `observable:AutonomousSystem` to represent that a subnet is declared by an AS. Similarly, we define the `dseco:is_part_of` property to enable representing `observable:IPAddress` to `observable:NetworkSubnet` relationships. Using transitivity of `dseco:hasAS` and `dseco:is_part_of`, we enable reasoning on `observable:IPAddress` to AS relationships. Finally, we define the `dseco:ZONE` class to enable reasoning about FQDNs of the same zone using the `dseco:is_in_zone` property. The `dseco:hasAS`, `dseco:is_part_of`, and `dseco:is_in_zone` properties were introduced to address gaps in UCO and support the DNS-KG use cases in Section III-A. For instance, these properties facilitate zone-based assessment, such as the *uc_complete_cleanup* case in Listing 2, by linking `dseco:ZONE` entities to organizational units in charge of the zone via the `dseco:managedBy` property. To represent organizational units, we reuse the `org:OrganizationalUnit` class from the ORG vocabulary.

C. Knowledge Graph Construction & Exploitation

In this work, we refer to the UNIX philosophy [16], [33] as guideline for designing the DNS-KG data processing architecture, which aims to achieve the audit and correction of DNS records in a timely manner while minimizing the need for specific or additional codebases and storage resources. For instance, using ontologies is a way to write the minimum amount of code by leveraging declarative techniques and

²²LDAP: Lightweight Directory Access Protocol.

²³<https://protege.stanford.edu/>

reusing open-source models from the community. Furthermore, the use cases in Section III-A suggest that a fundamental goal of this work is to answer queries without the need for a backend (e.g. a lambda architecture to feed a graph database [29])²⁴ or frontend (e.g. a graphical user interface combining data rendering and anomaly detection tools [28]), but to only focus on ontologies and inferences. Therefore, in what follows, we provide the details of a two-fold process: a *knowledge graph construction* step (i.e. instantiate the DSecO ontology in a simple text file through data wrangling of many managed DNS zones and other sources) followed by a *DNS configuration assessment* step (i.e. inferring on the resulting knowledge graph by querying directly on the corresponding text file and discard the file once finished). Overall, we assume that no memory or long-term retention of the data is required for the current DNS-KG objectives, but the use cases results (e.g. “UC #1 Inference Response” in Figure 1) and their explicability (i.e. the intrinsic explainability of the inference process based on the DSecO ontology and the logical form of the use case queries) are essential.

a) *Knowledge graph construction*: For this step, we consider a declarative data transformation approach to enable a knowledge graph construction process that is easily auditable, extensible, and platform-independent. We leverage the RML [3] vocabulary to implement a set of data transformation rules (i.e. TriplesMaps) for DNS configuration dumps of managed zones gathered from the technical ecosystem at Orange, and subnet/AS relationships dumps gathered from the BGPKIT public API.²⁵ Listings 11 & 12 provide an example of such transformation rule for the BGPKIT source and resulting knowledge graph, respectively.

Given the potential severe consequences of incorrect *DNS modifications* (e.g. deleting the wrong DNS entry can disrupt services), we employ a version control system for the transformation rules and adopt a CI/CD²⁶ approach for knowledge graph construction. This includes calling an RML engine (like RMLMapper²⁷ or BURP [14]), applying software engineering practices such as unit testing with SPARQL queries for posterior validation of the construction process at the knowledge graph level, and automated documentation generation using Ontopsy [36] for visual inspection of RDF models.

The resulting knowledge graph is saved to a file for further processing, including syntax and structure validation (which was mentioned earlier under the term “unit test”), enrichment with SPARQL UPDATE queries (Listings 13 and 14), and inference with a combination of SPARQL UPDATE, ASK, and SELECT queries (further discussed below as the *DNS configuration assessment* step).

Regarding unit tests, we implement a set of tests following the BDD approach by defining test cases in Gherkin linked to SPARQL queries (Listings 15 and 16 for execution by the

Listing 11 Snippet of data transformation rules using the RML vocabulary to materialize AS & subnet entities and relationships from a BGPKIT dump. An `adhoc dseco:hasAS` property is used to assert subnet/AS relationships before joining with DNS configuration entities from other sources and filtering out irrelevant AS & subnet entities.

```
1 @prefix rr: <http://www.w3.org/ns/r2rml#>.
2 @prefix rml: <http://semweb.mmlab.be/ns/rml#>.
3 @prefix dseco: <https://w3id.org/dseco/ontology/>.
4 @prefix observable: <https://ontology.unifiedcyberontology.org/uco/observable/>.
5 @base <https://w3id.org/dseco/rml/>.
6
7 <LS_BGPKIT_ASN> a rml:LogicalSource;
8   rml:source "/tmp/pfx2as-latest.json";
9   rml:referenceFormulation ql:JSONPath;
10  rml:iterator "$[*]".
11
12 <TM_ASN> a rr:TriplesMap;
13   rml:logicalSource <LS_BGPKIT_ASN>;
14
15   rr:subjectMap [
16     rr:template "https://w3id.org/dseco/kg/SUBNET_{prefix}";
17     rr:class observable:NetworkSubnet;
18     rr:class owl:NamedIndividual ];
19
20   rr:predicateObjectMap [
21     rr:predicate dseco:hasAS;
22     rr:objectMap [
23       rr:template "https://w3id.org/dseco/kg/AS_{asn}" ]].
```

Listing 12 Example of triple generated for the BPGKIT source from the RML in Listing 11.

```
1 <https://w3id.org/dseco/kg/SUBNET_193.252.148.0%2F24>
2   rdf:type owl:NamedIndividual;
3   rdf:type observable:NetworkSubnet;
4   dseco:hasAS <https://w3id.org/dseco/kg/AS_8891>.
```

Listing 13 Example of a SPARQL enrichment query that inserts `dseco:isinOurSubnet` facts based on `observable:IPAddress` entities being part of specific subnets.

```
1 PREFIX dseco: <https://w3id.org/dseco/ontology/>
2 PREFIX observable: <https://ontology.unifiedcyberontology.org/uco/observable/>
3
4 INSERT { ?ip dseco:isinOurSubnet true. }
5 WHERE {
6   ?ip a observable:IPAddress;
7     dseco:is_part_of ?SUBNET.
8
9   VALUES ?SUBNET {
10     <https://w3id.org/dseco/kg/SUBNET_193.XXX.YYY.0/24>
11     <https://w3id.org/dseco/kg/SUBNET_193.XXX.ZZZ.0/24>
12     ...
13     <https://w3id.org/dseco/kg/SUBNET_2a01:zzzz::/40> } }
```

Listing 14 Example of a SPARQL enrichment query that inserts `mykg:FQDN_resolving` facts in the knowledge graph based on the transitive closure of a CNAME chain using a property path.

```
1 PREFIX mykg: <https://w3id.org/dseco/kg/>
2
3 INSERT { ?Fqdn_orig a mykg:FQDN_resolving. }
4 WHERE {
5   ?Fqdn_orig a dseco:FQDN;
6     dseco:is_CNAME_of+ ?Fqdn_end.
7   ?Fqdn_end dseco:is_A_to ?_IP. }
```

²⁴A first approach for the DNS-KG work was to use a Neo4J database (<https://neo4j.com/>), but it led to writing specific code and design a specific data model, thus distancing us from the community.

²⁵BGPKIT, `pfx2as`: <https://data.bgpkit.com/pfx2as>

²⁶CI/CD: Continuous Integration/Continuous Delivery.

²⁷<https://github.com/RMLio/rmlmapper-java>

CI/CD automation during each code and knowledge graph construction process update.

This involves creating baseline datasets and corresponding knowledge graphs for each test case. We use the *behave* library²⁸ to parse unit tests and verify expected results. We also implement *LibOntology*, a Python package that facilitates ontology-related operations from the command line (e.g. querying, updating, validating, converting, and reasoning). The *LibOntology* code is available as open source at <https://w3id.org/dseco/tools/libontology>, along with its documentation.

Listing 15 Example of a unit test in Gherkin syntax to validate the knowledge graph construction step for the `dseco:ZONE` concepts against a baseline dataset.

```
1 Given that the KG is loaded
2   and the reasoner applied
3 When I run SPARQL query located at
4   queries/test_example.org_entries/test.sparql
5   to gather entries with ZONE is example.org
6 Then I should obtain the results from
7   file queries/test_example.org_entries/test.csv
```

Listing 16 Example of a unit test SPARQL query to validate the knowledge graph construction process in relation to the Gherkin model in Listing 15.

```
1 SELECT ?name
2 WHERE {
3   ?fqdn dseco:is_in_zone mykg:ZONE_example.org;
4   dseco:hasName ?name.
5 } ORDER BY ?name
```

b) DNS configuration assessment: We approach decision making on the DNS configuration as a classification task over the knowledge graph entities. We specifically consider that the *DNS & Network Configuration Update* actions (Figure 1) resulting from the assessment process (i.e. *configuration cleanup*, *information*, and *vulnerability analysis* as outlined in Section III-A) are defined based on the existence of entities that meet the axiomatic criteria for each use case.

We observe that the aforementioned testing framework developed for the *knowledge graph construction* step aligns with the goals of *DNS configuration assessment*. Therefore, we consider reusing the same toolset and coupling each use case Gherkin model from Section III-A with its translation into SPARQL queries to run on the resulting knowledge graph.²⁹ For example, `uc_ip_hijacking` in Listing 9 translates as the SPARQL query in Listing 17.

Assessment queries may require knowledge graph enrichment, such as the `uc_ip_hijacking` example in Listing 17 which identifies if a given IP address belongs to a collection of managed subnets. To achieve this, we add ad-hoc statements during the *knowledge graph construction* step via SPARQL UPDATE queries to benefit multiple use cases. For instance, `dseco:isinOurSubnet` statements are inserted into the

Listing 17 Example of an assessment case SPARQL query for `uc_ip_hijacking` related to the Gherkin model in Listing 9.

```
1 SELECT ?nameOrig ?IP_End
2 WHERE {
3   ?Fqdn_orig dseco:is_CNAME_of+ ?Fqdn_end.
4   FILTER NOT EXISTS {
5     ?Fqdn_end dseco:is_CNAME_of ?_OtherFQDN }
6
7   ?Fqdn_orig dseco:hasName ?nameOrig;
8   dseco:is_in_zone ?Zorig.
9   ?Zorig dseco:hasName "example.org.".
10  ?Fqdn_end dseco:hasName ?nameEnd;
11  dseco:is_A_to ?IP_End.
12
13  FILTER NOT EXISTS { # The interesting part of the query
14    ?IP_End dseco:isinOurSubnet "true"^^xsd:boolean. }
15 } ORDER BY ?nameOrig ?nameEnd # For determinism
```

knowledge graph after data wrangling using the query in Listing 13.

Assessment queries may also require knowledge beyond the data wrangling domain, such as the `uc_domain_hijacking` example in Listing 18, which uses organizational units responsible for DNS zones. To achieve this, we incorporate a *Domain Specific Individuals* knowledge base (Figure 1) during the *knowledge graph construction* step by concatenating a flat knowledge graph file. For instance, this includes detailed entities like `mykg:ORGU_HQ` and `mykg:ORGU_Affiliate` (Listing 18), facilitating the execution of `uc_domain_hijacking`.

Listing 18 Example of a assessment case SPARQL query for `uc_domain_hijacking` in relation to the Gherkin model in Listing 8.

```
1 SELECT ?nameOrig ?name_intermediate_or_end
2 WHERE {
3   ?Fqdn_orig dseco:is_CNAME_of+ ?Fqdn_intermediate_or_end.
4
5   ?Fqdn_orig dseco:hasName ?nameOrig;
6   dseco:is_in_zone ?Zorig.
7   ?Zorig dseco:hasName "example.org."^^xsd:string.
8
9   ?Fqdn_intermediate_or_end
10  dseco:hasName ?name_intermediate_or_end;
11  dseco:is_in_zone ?zone_intermediate_or_end.
12
13  # Exclude if managed by corporate headquarters
14  FILTER NOT EXISTS {
15    ?zone_intermediate_or_end
16    dseco:managedBy mykg:ORGU_top. }
17
18  # Exclude if managed by affiliates
19  FILTER NOT EXISTS {
20    ?zone_intermediate_or_end
21    dseco:managedBy mykg:ORGU_Affiliate. } }
22 # For determinism
23 ORDER BY ?nameOrig ?name_intermediate_or_end
```

Overall, this approach enables the decoupling of the ontology and assessment queries from the application context, allowing each DNS-KG user to customize the assessment process by adding, updating, or removing use cases and domain-specific individuals. We foresee that by applying these principles (i.e. graph enrichment, knowledge base concatenation, assessment parametrization), the DNS-KG could not only identify inconsistencies and non-compliances but also mitigate risks through a what-if approach by incorporating fictive entities (e.g. potential misconfigurations or cybersecurity

²⁸<https://github.com/behave/behave>

²⁹Queries for the use cases in Section III-A are available open-source at <https://w3id.org/dseco/>

TABLE I
DNS-KG DATASET STATISTICS

	Toy example	Real-world
Class	Count	Count
dseco:FQDN	43	22'404
dseco:ZONE	7	297
observable:IPAddress	12	7'817
observable:IPAddressFacet	12	7'817
observable:IPv4Address	11	7'079
observable:IPv6Address	1	738
observable:NetworkSubnet	5	13
observable:AutonomousSystem	2	162
observable:AutonomousSystemFacet	2	162
mykg:FQDN_resolving	17	7'803
mykg:FQDN_orange_resolving	0	4'036
Property	Count	Count
dseco:hasName	50	21'905
dseco:hasAS	5	6'339
dseco:hasOrgu	2	162
dseco:is_A_to	12	12'971
dseco:is_AAAA_to	0	873
dseco:is_CNAME_of	27	9'838
dseco:is_in_zone	41	21'608
dseco:is_part_of	11	906
dseco:managedBy	4	297
uco:asHandle	2	162
uco:addressValue	12	7'817
dseco:isinOurSubnet	6	906
dseco:is_CNAME_of_CNAME_of	15	5'130
mykg:isUsingOurCDN	0	42

vulnerabilities) to explore use cases and understand associated risks, in accordance with the method proposed in [52], thereby facilitating proactive risk management.

IV. EXPERIMENTS

In this section, we present the experiments conducted to evaluate the DNS-KG approach as defined in Section III. We begin by presenting the experimental setup in Section IV-A, which involves executing the *knowledge graph construction* and *DNS configuration assessment* steps on a toy example dataset and a real-world dataset based on Orange data.

We then present the results of these executions (in terms of execution time and statistics on the queries) in Section IV-B, followed by a discussion of these results in Section IV-C. The queries used for these experiments and the generated dataset are available at <https://w3id.org/dseco/>. Due to confidentiality, the real-world dataset is not made public. Table I summarizes the characteristics of the two datasets, and Table II summarizes the results of the *DNS configuration assessment* step for both the toy example and real-world dataset.

A. Experimental Setup

Whether for the toy example dataset or the real-world dataset, we performed the *knowledge graph construction* and *DNS configuration assessment* steps as follows: 1) in a dedicated GitLab project, we initiate a *base DNS-KG graph file* that imports the DSecO vocabulary (Listing 19); 2) start the data wrangling process (Listing 20), which loops over a set of sources (bgpkit, data, janus) and for each source loops over RML mappings and concatenates the resulting set of triples to the aforementioned base DNS-KG graph file; 3)

Listing 19 Base DNS-KG file with import statement for the DSecO vocabulary.

```
1 <http://dns-kg> rdf:type owl:Ontology ;
2 owl:imports <https://w3id.org/dseco/ontology/dseco-
  latest.ttl> .
```

validate the DNS-KG file syntax through *LibOntology* (i.e. using the `libontology-validate` method); 4) configure a filesystem using the queries/`<UseCaseName>/` scheme to store Gherkin models (Section III-A) and assessment SPARQL queries according to the set of

Listing 20 Command line for knowledge graph construction and processing logs, highlighting three data sources (bgpkit, data, and janus), twelve RML rule files (each implementing one TripleMap), and the number of individuals generated by each rule set for the real-world dataset.

```
1 ./ontology_builder_rml.py --sourcesfolder
  ontology_builder_rml/sources --stats > /tmp/onto.ttl
2
3 [*] Process: bgpkit/mapping_rml/mapping_asn_prefix.ttl
4 [*] Individuals : 13
5 [*] Process: janus/mapping_rml/mapping_cname.ttl
6 [*] Individuals : 9832
7 [*] Process: janus/mapping_rml/mapping_a_record.ttl
8 [*] Individuals : 11616
9 [*] Process: janus/mapping_rml/mapping_orgu_dynamic.ttl
10 [*] Individuals : 151
11 [*] Process: janus/mapping_rml/mapping_ipv4.ttl
12 [*] Individuals : 7079
13 [*] Process: janus/mapping_rml/mapping_ipv6.ttl
14 [*] Individuals : 738
15 [*] Process: janus/mapping_rml/mapping_orgu_asn.ttl
16 [*] Individuals : 145
17 [*] Process: janus/mapping_rml/mapping_aaaa_record.ttl
18 [*] Individuals : 510
19 [*] Process: data/mapping_rml/mapping_zone_someoneElse.ttl
20 [*] Individuals : 270
21 [*] Process: data/mapping_rml/mapping_zone_top.ttl
22 [*] Individuals : 14
23 [*] Process: data/mapping_rml/mapping_zone_int.ttl
24 [*] Individuals : 13
25 [*] Process: data/mapping_rml/mapping_orgu_static.ttl
26 [*] Individuals : 3
```

use cases to run; 5) enrich the DNS-KG using a set of SPARQL UPDATE queries (e.g. Listings 13 & 14) through *LibOntology* (i.e. using the `libontology-update` method); 6) run the use cases against the DNS-KG file using a Bash for loop³⁰ to automatically discover implemented queries/`<UseCaseName>/test.sparql` SPARQL queries and call these through *LibOntology* (i.e. using the `libontology-query` method embedded into a Bash time command³¹ to measure the query execution time); 7) save the output and execution time of each assessment SPARQL query into separate text files and make them available for further analysis (e.g. identification of misconfiguration patterns in the dataset w.r.t. query responses, statistical comparison of the query execution times w.r.t. query complexity [23], and DNS and network misconfiguration remediation by NetOps/SecOps); 8) to establish a baseline for query execution times, we repeat steps #6 and #7 using a semantic graph database (i.e. a local GraphDB³² instance) by loading

³⁰<https://www.gnu.org/software/bash/>

³¹<https://www.gnu.org/software/time/>

³²<https://graphdb.ontotext.com/>

TABLE II
QUERY EXECUTION TIME

	Query	Toy example		Real-world	
		libontology [s]	GraphDB [s]	libontology [s]	GraphDB [s]
a)	uc_complete_cleanup	0.15	0.10	7.20	0.80
b)	uc_complete_cleanup_not_resolving	0.16	0.10	20.77	1.90
c)	uc_loop	0.15	0.10	4.63	0.10
d)	uc_ultimately_do_not_resolve_to_an_ip_address	0.16	0.10	5.30	0.10
e)	uc_entries_using_our_CDN	0.15	0.10	5.07	0.10
f)	uc_resolving_to_a_single_IP_address	0.15	0.10	6.00	0.20
g)	uc_domain_hijacking	0.16	0.10	5.40	0.20
h)	uc_ip_hijacking	0.15	0.10	5.60	0.10
i)	uc_ip_hijacking_filter	0.17	0.10	5.79	0.10

the DNS-KG file into the database and running the queries against its SPARQL endpoint.

B. Results

The results presented below – particularly the timing measures – stem from the execution of the methods and tools described in Section IV-A on a MacBook M1 Pro 2021 (32 Gbyte RAM) test machine. Table I summarizes, for each dataset (i.e. toy example & real-world), the characteristics of the DNS-KG resulting from the *knowledge graph construction* step on which the *DNS configuration assessment* step was conducted. It includes the number of entities per class and the number of instantiated properties, grouped into facts derived from data transformation (e.g. `dseco:FQDN`, `dseco:hasName`) and those computed (e.g. `mykg:FQDN_resolving`, `dseco:isinOurSubnet`). Similarly, Table II reports on the execution time of the different queries for the *DNS configuration assessment* step. We break down the results for each dataset below.

a) *Toy example dataset*: The dataset consists of 43 DNS records and contains 31 hand-crafted non-compliances.³³ The execution of the `libontology-validate` step takes $t_{validate} = 0.10$ s. The execution of the `libontology-query` step takes an average $\bar{t}_{query} = 0.16$ s per SPARQL query (this amounts to a total of $t_{queries} = 1.40$ s for the entire process using a sequential execution scheme),³⁴ with correct detection of all 31 non-compliant configuration cases. Executing the same queries on the GraphDB SPARQL endpoint results in a consistent $t_{query} = 0.10$ s per query and also allows for the detection of all non-conformities. files.

b) *Real-world dataset*: For this dataset, 30'375 DNS records were used for constructing the DNS-KG, with 4'000 of these records being internally managed (`ORGint` in Eq. 1) and 21'000 from affiliates (`ORGtop` in Eq. 1). The knowledge graph construction step (i.e. data fetching & RML-based data transformation) takes $t_{wrangling} = 43$ s. The resulting knowledge graph is roughly 10 Mbyte in size. The execution of the `libontology-validate` step takes $t_{validate} = 4.90$ s. The execution of the `libontology-query` shows a total

process duration of $t_{queries} = 65.76$ s using a sequential execution scheme (i.e. average $\bar{t}_{query} = 65.76/9 \simeq 7.3$ s per SPARQL query).³⁵ Over 547 non-compliances were reported and subsequently forwarded to a SecOps team through a ticketing system for further analysis and remediation.³⁶ Among these non-compliances, the top three use cases in terms of operational complexity for carrying out the remediations and associated cybersecurity risks break down as follows:

- `uc_complete_cleanup`: a total of 347 non-compliances were directly remediated with the elimination of redundant or incorrect DNS records in less than two days of work, as they relate to internally managed records and SecOps has high confidence in the remediation actions based on the use case definition.
- `uc_domain_hijacking`: at least 100 non-compliances related to the front-facing records were reviewed by experts, as the question posed involves identifying entries that pass through an intermediate organizational unit. Two were then remediated by triggering remediation operations with third-party teams managing these records. Another non-compliance, similar to the NS case described in [4], was also detected using this use case and corrected promptly.
- `uc_ip_hijacking`: in the same way as for `uc_domain_hijacking`, at least 100 non-compliances were reviewed, of which one was remediated through the intervention of third-party teams.

C. Discussion

From the results presented in the previous paragraph, the detection of 100% of cases in the toy example dataset demonstrates the effectiveness of the proposed architecture. Additionally, the handling of over 547 non-compliance reports in the real-world dataset, along with the rapid identification of 350 cases needing remediation, confirms the operational relevance of the DNS-KG approach proposal.

We note that before the implementation of the DNS-KG proposal, the *DNS configuration assessment* (i.e. auditing and cleaning tasks) was rarely conducted in such depth, if at all, by the involved SecOps team. This allows us to conclude that the overall DNS-KG approach improves operational efficiency

³³Non-compliances details are available online in https://w3id.org/dseco/evaluation/dseco-<release>/queries/uc_<name>/test.csv

³⁴Parallel execution of SPARQL queries is possible, enabling shorter overall assessment times.

³⁵As for the toy example, parallel execution of SPARQL queries is possible, enabling shorter overall assessment times.

³⁶Specific cases are undisclosed to protect Orange's interests.

(from nearly infinite operation time to approximately 1.5 days for internally managed records) by instilling confidence in the corrective actions to be carried out over a large number of records. Beyond operational efficiency, we also assessed learning and adoption among the SecOps team. Using the DSecO online resources – notably the ontology documentation with examples and the RML & SPARQL constructs – operation engineers built their knowledge graph and ran queries within a few hours. Although creating SPARQL queries from scratch was initially challenging, Gherkin models and their connection to existing queries enabled quick adaptation for user-defined use cases. Furthermore, we observed that every instance of non-compliance detected could be transformed into unit tests for DSecO, as part of a continuous improvement cycle, thereby strengthening confidence in the solution.

The operational complexity observed in the case of the real-world dataset (i.e. remediating `uc_complete_cleanup` versus `uc_domain_hijacking` and `uc_ip_hijacking`) raises questions about the scope of the *knowledge graph construction* (KGC) step (i.e. *are we missing some external data that could help provide a more reliable diagnosis? is the KGC step accurate regarding real-world data?*) in relation to the “almost zero doubt” goal for *DNS configuration assessment*. Therefore, integrating additional non-managed data & auditable KGC with data quality checks should be further explored.

We also note that, based on the execution times in Table II, the DNS-KG approach, while not real-time (focused on producing easily comprehensible and explainable results), exhibits highly variable query execution durations for both LibOntology and GraphDB, with similar trends observed for both. To begin with, comparing the timings of LibOntology and GraphDB questions about the principle of minimizing the need for specific or additional codebases and storage resources, as discussed in Section III-C, and provides an indication of the proportion and actual duration of query execution time within the process. Indeed, with GraphDB, the initial loading time of the graph is bypassed in the measurements in Table II since the graph is loaded once at the beginning of the process, whereas LibOntology requires loading the graph into memory for each query execution. Taking this further, comparing measurements across the use cases also raises questions about the relationship between the size of the DNS-KG (Table I), the formulation of use cases into SPARQL queries (i.e. *could the queries be rewritten for lower algorithmic complexity?* [23], [47]), and the inference techniques used (e.g. *would SHACL or SWRL be more efficient regarding the expressiveness expected by the use cases?* [2], [5]). For example, the use case `uc_ip_hijacking` involves adding `dseco:isInOurSubnet true` facts using a SPARQL `VALUE` statement (Listing 13). Similarly, the `uc_entries_using_our_CDN` implementation currently combines a `FILTER` statement with a regular expression to match a `dseco:hasName` value pattern on a `dseco:is_CNAME_of` property path. While functional, this can be optimized for processing performance (time complexity is $O(n)$ for `VALUE` & `FILTER` statements and $O(n)$ for

DFA-compiled regular expression against a string [47]) and thoroughness (keeping subnet entities and FQDN name values updated). Incorporating network topology data into the knowledge graph could improve network element selection efficiency for the `uc_entries_using_our_CDN` case through graph traversal and reduce regex reliance, though it would require linking DSecO to third-party vocabularies to accommodate new data objects (e.g. [30], [39]).

V. CONCLUSION AND FUTURE WORK

In this work, we aimed to simplify Domain Name System (DNS) administration, particularly in systems where DNS configuration information includes a large number of records, originates from heterogeneous sources, and is managed by different organizational units.

We initially hypothesized that an explicit graph representation could reflect the chaining nature of DNS records and provide formal guarantees, notably in terms of a shared understanding of the unified view of DNS records among stakeholders, while also establishing actionable, explainable, and shareable DNS administration rules.

Based on this idea, we proposed the “DNS-KG” approach, which firstly involved conducting an analysis of DNS administration operations with SecOps experts from Orange. This first step allowed us to define nine key use cases and formalize them using a Behavior-Driven Development (BDD) approach. The resulting structuring of the use cases proved beneficial for establishing consensus among the experts and facilitating the translation of the use cases into actionable rules. We focused on auditing DNS records from diverse data sources by creating an RDF knowledge graph structured with the DSecO vocabulary, an open-source RDFS/OWL ontology we implemented and make available at <https://w3id.org/dseco>. This implementation derives from the use cases defined with the SecOps experts and adheres to DNS standards defined in RFCs. We used a declarative approach based on the RML vocabulary for knowledge graph construction. DNS configuration assessment is performed with SPARQL queries corresponding to the nine use cases. The evaluation on a toy example dataset confirmed the effective functioning of the proposed DNS-KG approach. Similarly, the evaluation on a real-world dataset highlighted the relevance of the DNS-KG approach in an industrial setting, enabling SecOps teams to confidently identify and resolve numerous non-compliant DNS configurations. It also indicated that the inference process performance during the DNS-KG assessment step could be enhanced by reducing the computational complexity of SPARQL queries through the incorporation of additional data during knowledge graph construction, rather than relying on SPARQL `FILTER` statements.

Based on these insights, we are considering future work along three axes. A first focus area is the *extension of the use cases*. This can be achieved by conducting interviews with a broader range of NetOps and SecOps experts, reviewing RFCs for configuration rules (e.g. RFC 1912 [6] on common DNS operational and configuration errors), examining research papers for configuration corner cases (e.g. the “zombie awakening” [15], TCB [45], and TsuNAME [12]

concepts), and analyzing incident tickets in relation to DNS-KG graph patterns [27]. Another approach is to tag DNS-KG assessment cases with STIX/DRM [5] concepts or MITRE ATT&CK techniques³⁷ to create a reusable rule catalog based on community-shared concepts. This tagging could also support downstream decision systems, such as ticketing systems, where risk calculus, remediation prioritization, and causal reasoning (e.g. on network reachability/connectivity incidents) could be assessed through the DNS-KG.

A second focus is the *extension of the input data range and DSecO vocabulary*. Indeed, we recognize that adding use cases involves broadening the DSecO inference scope and integrating more data sources (e.g. network topology, details of Docker instances, CVEs) for knowledge graph construction. This entails extending the DSecO ontology with new concepts and relationships, such as NS, CAA, TXT, DNAME, NSEC3, RRSIG, MX, and other DNS record types. It may also involve strengthening the linking of DSecO to other vocabularies (e.g. UCO [56] & D3FEND [41] for cyber threat intelligence; NORIA-O [30] & SEON [35] for information system topology and operations; HTTPinRDF [21] for L7 details). For instance, reasoning about HTTP redirect responses [43] as a graph within a DNS-KG instance could assist NetOps & SecOps in tracing the causes of network service performance issues or cyberattacks. Similarly, “what if scenarios” reasoning on potential service impact of changing an IP address or CNAME could also assist NetOps & SecOps before planned network changes.

Finally, a third focus area is the *improvement of the technical aspects of DNS-KG*. Leveraging data provenance [34], [46] at the DNS-KG level could help monitor data quality and inferences, increasing trust in the graph’s exploitation. In cases of incomplete DNS data, inserting blank nodes to ensure use case functionality is a potential solution to explore. Additionally, investigating stream processing for the DNS-KG approach – with RML rules for graph construction and SPARQL queries remaining unchanged thanks to already existing tools [11], [29] – may enable real-time applications. Although the current use cases do not require stream processing, this could facilitate higher-level use cases in network change or fault management. In a similar vein, integrating the DNS-KG approach into Infrastructure-as-Code systems may prevent risky DNS deployments, such as fast-failing CI/CD pipelines responsible for DNS configuration. Lastly, improving graph enrichment and assessment query performance might involve comparing the execution time of current DNS-KG SPARQL queries with their SWRL [5] and SHACL [2] equivalents. Since SWRL performance degrades with increased dependencies [2], this comparison can help establish an indicator for expected expressiveness in DNS-KG use cases and guide the selection of preferred inference techniques.

ACKNOWLEDGMENT

The authors would like to thank Maxime Jerome and Dao-Lam Trinh-Huu-Phap for their support on preparatory work and for providing access to specific datasets. They appreciate

Dialog³⁸ for helping them initiate this work, and also would like to express their gratitude to Antoine Cawet for his dedication and efforts in implementing and improving the DSecO Project. They would also like to thank Yoan Chabot for reviewing and providing feedback on the preliminary version of this article and IEEE/ACM TRANSACTIONS ON NETWORKING editorial committee and reviewers for their detailed analysis and recommendations for improving the article.

REFERENCES

- [1] A. Hogan et al., “Knowledge graphs,” *ACM Comput. Surv.*, vol. 54, no. 4, pp. 1–37, May 2022, doi: [10.1145/3447772](https://doi.org/10.1145/3447772).
- [2] A. Guittoum, F. Aïssaoui, S. Bolle, F. Boyer, and N. De Palma, “Inferring threatening IoT dependencies using semantic digital twins toward collaborative IoT device management,” in *Proc. 38th ACM/SIGAPP Symp. Appl. Comput.*, Mar. 2023, pp. 1732–1741, doi: [10.1145/3555776.3578573](https://doi.org/10.1145/3555776.3578573).
- [3] A. Dimou, “RML: A generic language for integrated rdf mappings of heterogeneous data,” in *Proc. Workshop Linked Data Web, LDOW Co-Located 23rd Int. World Wide Web Conf. (WWW)*, vol. 1184, 2014.
- [4] B. Krebs. (2025). *MasterCard DNS Error Went Unnoticed for Years*. [Online]. Available: <https://krebsonsecurity.com/2025/01/mastercard-dns-error-went-unnoticed-for-years/>
- [5] C. Sánchez-Zas, V. A. Villagrà, M. Vega-Barbas, X. Larriva-Novo, J. I. Moreno, and J. Berrocal, “Ontology-based approach to real-time risk management and cyber-situational awareness,” *Future Gener. Comput. Syst.*, vol. 141, pp. 462–472, Apr. 2023, doi: [10.1016/j.future.2022.12.006](https://doi.org/10.1016/j.future.2022.12.006).
- [6] D. Barr, *Common DNS Operational and Configuration Errors*, document RFC 1912, 1996, doi: [10.17487/RFC1912](https://doi.org/10.17487/RFC1912).
- [7] D. Liu, S. Hao, and H. Wang, “All your DNS records point to U.S.: Understanding the security threats of dangling DNS records,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, Oct. 2016, pp. 1414–1425, doi: [10.1145/2976749.2978387](https://doi.org/10.1145/2976749.2978387).
- [8] D. Liu, Z. Li, K. Du, H. Wang, B. Liu, and H. Duan, “Don’t let one rotten apple spoil the whole barrel: Towards automated detection of shadowed domains,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, Oct. 2017, pp. 537–552, doi: [10.1145/3133956.3134049](https://doi.org/10.1145/3133956.3134049).
- [9] D. Brickley and R. V. Guha. (2014). *RDF Schema 1.1*. W3C. [Online]. Available: <https://www.w3.org/TR/rdf-schema/>
- [10] D. Reynolds. (2014). *The Organization Ontology*. W3C Recommendation, W3C. [Online]. Available: <https://www.w3.org/TR/vocab-org/>
- [11] D. F. Barbieri, D. Braga, S. Ceri, E. Della Valle, and M. Grossniklaus, “C-SPARQL: SPARQL for continuous querying,” in *Proc. 18th Int. Conf. World Wide Web*, Apr. 2009, pp. 1061–1062, doi: [10.1145/1526709.1526856](https://doi.org/10.1145/1526709.1526856).
- [12] D. Kontokostas et al., “Test-driven evaluation of linked data quality,” in *Proc. 23rd Int. Conf. World Wide Web*, Apr. 2014, pp. 747–758, doi: [10.1145/2566486.2568002](https://doi.org/10.1145/2566486.2568002).
- [13] J. C. Klensin, *Simple Mail Transfer Protocol*, document RFC 5321, 2008, doi: [10.17487/RFC5321](https://doi.org/10.17487/RFC5321).
- [14] D. Van Assche and C. Debruyne, “BURPing through RML test cases,” in *Proc. 5th Int. Workshop Knowl. Graph Construct. Co-Located With 21st Extended, Semantic Web Conf. (ESWC)*, vol. 3718, Hersonissos, Greece, May 27, 2024.
- [15] E. Alowaisheq, S. Tang, Z. Wang, F. Alharbi, X. Liao, and X. Wang, “Zombie awakening: Stealthy hijacking of active domains through DNS hosting referral,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, Oct. 2020, pp. 1307–1322, doi: [10.1145/3372297.3417864](https://doi.org/10.1145/3372297.3417864).
- [16] E. S. Raymond, *The Art of UNIX Programming: With Contributions From Thirteen UNIX Pioneers, Including its Inventor, Ken Thompson* (Addison-Wesley Professional Computing Series). Reading, MA, USA: Addison-Wesley, 2008.
- [17] G. Akiwate et al., “Retroactive identification of targeted DNS infrastructure hijacking,” in *Proc. 22nd ACM Internet Meas. Conf.*, Oct. 2022, pp. 14–32, doi: [10.1145/3517745.3561425](https://doi.org/10.1145/3517745.3561425).
- [18] G. C. M. Moura, S. Castro, J. Heidemann, and W. Hardaker, “TsuNAME: Exploiting misconfiguration and vulnerability to DDoS DNS,” in *Proc. 21st ACM Internet Meas. Conf.*, Nov. 2021, pp. 398–418, doi: [10.1145/3487552.3487824](https://doi.org/10.1145/3487552.3487824).

³⁷<https://attack.mitre.org/techniques/enterprise/>

³⁸<https://www.dialog.com/>

- [19] I. Horrocks, P. F. Patel-Schneider, H. Boley, S. Tabet, B. Grosz, and M. Dean, "SWRL: A semantic web rule language combining OWL and RuleML," Nat. Res. Council Canada, Netw. Inference, Stanford Univ., Stanford, CA, USA, W3C Member Submission, Tech. Rep. SUBM-SWRL-20040521, 2004.
- [20] J. Hodges, C. Jackson, and A. Barth, *HTTP Strict Transport Security (HSTS)*, document RFC 6797, 2012, doi: [10.17487/RFC6797](https://doi.org/10.17487/RFC6797).
- [21] J. Koch, C. A. Velasco, and P. Ackermann. (2017). *HTTP Vocabulary in RDF 1.0*. W3C Working Group Note, W3C. [Online]. Available: <https://www.w3.org/TR/HTTP-in-RDF10>
- [22] J. A. Hawkinson and T. J. Bates, *Guidelines for Creation, Selection, and Registration of an Autonomous System (AS)*, document RFC 1930, 1996, doi: [10.17487/RFC1930](https://doi.org/10.17487/RFC1930).
- [23] J. Pérez, M. Arenas, and C. Gutierrez, "Semantics and complexity of SPARQL," *ACM Trans. Database Syst.*, vol. 34, no. 3, pp. 1–45, Aug. 2009, doi: [10.1145/1567274.1567278](https://doi.org/10.1145/1567274.1567278).
- [24] K. Kurniawan, E. Kiesling, D. Winkler, and A. Ekelhart, "The ICS-SEC KG: An integrated cybersecurity resource for industrial control systems," in *The Semantic Web—ISWC 2024*. Cham, Switzerland: Springer, 2025, doi: [10.1007/978-3-031-77847-6_9](https://doi.org/10.1007/978-3-031-77847-6_9).
- [25] K. Hu, H. Du, and Y. Wang, "Heracles: A novel state-based distributed verification framework for DNS configurations," in *Proc. SIGCOMM Workshop Formal Methods Aided Netw. Operation*, Aug. 2024, pp. 27–32, doi: [10.1145/3672199.3673890](https://doi.org/10.1145/3672199.3673890).
- [26] K. Borgolte, T. Fiebig, S. Hao, C. Kruegel, and G. Vigna, "Cloud strife: Mitigating the security risks of domain-validated certificates," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, doi: [10.14722/ndss.2018.23327](https://doi.org/10.14722/ndss.2018.23327). [Online]. Available: <https://www.ndss-symposium.org/ndss2018/acceptedpapers/>
- [27] L. Tailhardat, R. Troncy, and Y. Chabot, "Leveraging knowledge graphs for classifying incident situations in ICT systems," in *Proc. 18th Int. Conf. Availability, Rel. Secur.*, Aug. 2023, pp. 1–9, doi: [10.1145/3600160.3604991](https://doi.org/10.1145/3600160.3604991).
- [28] L. Tailhardat, Y. Chabot, A. Py, and P. Guillemette, "NORIA UI: Efficient incident management on large-scale ICT systems represented as knowledge graphs," in *Proc. 19th Int. Conf. Availability, Rel. Secur.*, Jul. 2024, pp. 1–10.
- [29] L. Tailhardat, Y. Chabot, and R. Troncy, "Designing NORIA: A knowledge graph-based platform for anomaly detection and incident management in ICT systems," in *Proc. 4th Int. Workshop Knowl. Graph Construct. (KGCW)*, vol. 3471, 2023, pp. 1–16.
- [30] L. Tailhardat, Y. Chabot, and R. Troncy, "NORIA-O: An ontology for anomaly detection and incident management in ICT systems," in *Proc. Semantic Web 21st Int. Conf. (ESWC)*, Hersonissos, Greece, May 2024, pp. 21–39, doi: [10.1007/978-3-031-60635-9_2](https://doi.org/10.1007/978-3-031-60635-9_2).
- [31] M. B. Salem and C. Wacek, "Enabling new technologies for cyber security defense with the ICAS cyber security ontology," in *Proc. 10th Conf. Semantic Technol. Intell., Defense, Secur.*, Fairfax VA, USA, 2015, pp. 1–8.
- [32] M. Poveda-Villalón, A. Fernández-Izquierdo, M. Fernández-López, and R. García-Castro, "LOT: An industrial oriented ontology engineering framework," *Eng. Appl. Artif. Intell.*, vol. 111, May 2022, Art. no. 104755, doi: [10.1016/j.engappai.2022.104755](https://doi.org/10.1016/j.engappai.2022.104755).
- [33] M. D. McIlroy, E. N. Pinson, and B. A. Tague, "UNIX time-sharing system: Foreword," *Bell Syst. Tech. J.*, vol. 57, no. 6, pp. 1899–1904, Jul. 1978. [Online]. Available: <http://archive.org/details/bstj57-6-1899>
- [34] M. Herschel, R. Diestelkämper, and H. Ben Lahmar, "A survey on provenance: What for? What form? What from?," *VLDB J.*, vol. 26, no. 6, pp. 881–906, Dec. 2017, doi: [10.1007/s00778-017-0486-1](https://doi.org/10.1007/s00778-017-0486-1).
- [35] M. Wüsch, G. Ghezzi, M. Hert, G. Reif, and H. C. Gall, "SEON: A pyramid of ontologies for software evolution and its applications," *Computing*, vol. 94, no. 11, pp. 857–885, Nov. 2012, doi: [10.1007/s00607-012-0204-1](https://doi.org/10.1007/s00607-012-0204-1).
- [36] M. Pasin. (2017). *Ontopsy: Python Library and Command-Line Interface for Inspecting and Visualizing RDF Models*. [Online]. Available: <https://github.com/lambdamusic/ontopsy>
- [37] M. Taghavi, "A knowledge graph to represent software vulnerabilities," M.S. thesis, Dept. Comput. Sci. Softw. Eng., Concordia Univ., Montreal, QC, Canada, 2023. [Online]. Available: <https://spectrum.library.concordia.ca/id/eprint/991794/>
- [38] M. Zhang. (2021). *Bgpkitt*. [Online]. Available: <https://bgpkitt.com/>
- [39] O. Corcho et al., "A high-level ontology network for ICT infrastructures," in *Proc. 20th Int. Semantic Web Conf. (ISWC)*, 2021, doi: [10.1007/978-3-030-88361-4_26](https://doi.org/10.1007/978-3-030-88361-4_26).
- [40] P. Mockapetris, *Domain Names—Implementation and Specification*, document RFC 1035, 1987, doi: [10.17487/RFC1035](https://doi.org/10.17487/RFC1035).
- [41] P. E. Kaloroumakis and M. J. Smith, "Toward a knowledge graph of cybersecurity countermeasures," MITRE Corporation, McLean, VA, USA, Tech. Rep. D3FEND Paper, 2021.
- [42] P. Hallam-Baker, R. Stradling, and J. Hoffman-Andrews, *DNS Certification Authority Authorization (CAA) Resource Record*, document RFC 8659, 2019, doi: [10.17487/RFC8659](https://doi.org/10.17487/RFC8659).
- [43] R. Fielding, M. Nottingham, and J. Reschke, *HTTP Semantics*, document RFC 9110, 2022, doi: [10.17487/RFC9110](https://doi.org/10.17487/RFC9110).
- [44] R. Sommesse, M. Jonker, R. van Rijswijk-Deij, A. Dainotti, K. C. Claffy, and A. Sperotto, "The forgotten side of DNS: Orphan and abandoned records," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops (EuroS&PW)*, Sep. 2020, pp. 538–543, doi: [10.1109/EUROSPW51379.2020.00079](https://doi.org/10.1109/EUROSPW51379.2020.00079).
- [45] V. Ramasubramanian and E. G. Sirer, "Perils of transitive trust in the domain name system," in *Proc. 5th ACM SIGCOMM Conf. Internet Meas. (IMC)*, 2005, p. 1, doi: [10.1145/1330107.1330152](https://doi.org/10.1145/1330107.1330152).
- [46] R. Fontugne, M. Tashiro, R. Sommesse, M. Jonker, Z. S. Bischof, and E. Aben, "The wisdom of the measurement crowd: Building the internet yellow pages a knowledge graph for the internet," in *Proc. ACM Internet Meas. Conf.*, Nov. 2024, pp. 183–198, doi: [10.1145/3646547.3688444](https://doi.org/10.1145/3646547.3688444).
- [47] Russ Cox. (2007). *Regular Expression Matching Can Be Simple and Fast*. [Online]. Available: <https://swtch.com/~rsc/regexp/regexp1.html>
- [48] S. Dietzold, "Generating RDF models from LDAP directories," in *Proc. SFSW Workshop Scripting Semantic Web*, vol. 135, 2005, pp. 1–7.
- [49] S. Chavez-Feria, R. Garca-Castro, and M. Poveda-Villalón, "Chowlk: From UML-based ontology conceptualizations to OWL," in *Semantic Web*, vol. 13261. Cham, Switzerland: Springer, 2022, pp. 338–352, doi: [10.1007/978-3-031-06981-9_20](https://doi.org/10.1007/978-3-031-06981-9_20).
- [50] S. K. R. Kakarla, R. Beckett, B. Arzani, T. Millstein, and G. Varghese, "GRooT: Proactive verification of DNS configurations," in *Proc. Annu. Conf. ACM Special Interest Group Data Commun. Appl., Technol., Architectures, Protocols Comput. Commun.*, New York, NY, USA, Jul. 2020, pp. 310–328, doi: [10.1145/3387514.3405871](https://doi.org/10.1145/3387514.3405871).
- [51] (2019). *SmartBear Software: Gherkin Syntax*. [Online]. Available: <https://cucumber.io/docs/gherkin/>
- [52] S. Wu, Y. Zhang, and W. Cao, "Network security assessment using a semantic reasoning and graph based approach," *Comput. Electr. Eng.*, vol. 64, pp. 96–109, Nov. 2017, doi: [10.1016/j.compeleceng.2017.02.001](https://doi.org/10.1016/j.compeleceng.2017.02.001).
- [53] S. S. Alqahtani, E. E. Eghan, and J. Rilling, "Tracing known security vulnerabilities in software repositories—A semantic web enabled modeling approach," *Sci. Comput. Program.*, vol. 121, pp. 153–175, Jun. 2016, doi: [10.1016/j.scico.2016.01.005](https://doi.org/10.1016/j.scico.2016.01.005).
- [54] W3C OWL Working Group. (2012). *OWL 2 Web Ontology Language*. W3C. [Online]. Available: <https://www.w3.org/TR/owl2-overview/>
- [55] W3C SPARQL Working Group. (2013). *SPARQL 1.1*. W3C. [Online]. Available: <https://www.w3.org/TR/sparql11-overview/>
- [56] Z. Syed, A. Padia, M. L. Mathews, T. Finin, and A. Joshi, "UCO: A unified cybersecurity ontology," in *Proc. AAAI Workshop Artif. Intell. Cyber Secur.*, 2016, pp. 195–202.

Didier Bringer received the Engineering degree in computer science from Polytech Montpellier, ISIM, France, in 1995, and the another Engineering degree in cybersecurity and formal knowledge representation from the Conservatoire National des Arts et Métiers (CNAM), France, in 2022. Currently, he is a full-time Cybersecurity Engineer with Orange Company, France. Prior to specializing in cybersecurity, he was a System Engineer and the IT Platform Manager for various companies. His research interests include system vulnerability analysis, risk management, case-based reasoning, and knowledge capture.

Lionel Tailhardat received the M.Sc. degree in telecommunication networks and broadband technologies from the Conservatoire National des Arts et Métiers (CNAM), France, in 2018, and the Ph.D. degree in data science and knowledge engineering from EURECOM, Sorbonne University, France, in 2024. Currently, he is a full-time Researcher with Orange Research. Prior to pursuing research, he held various positions in network operations and monitoring system architecture. His research interests include distributed and dynamic systems, data science and graph theory, coding schemes, and high-performance computing.